



OSTİM TECHNICAL UNIVERSITY FACULTY OF ENGINEERING CAPSTONE PROJECT REPORT

PROJECT TITLE

Project Team

Student Full Name

Student ID

Project Advisor:

Advisor Title and Full Name

Undergraduate Project

Department Name

Course Code and Course Title

March 2026

APPROVAL

This undergraduate capstone project, entitled “.....,”
prepared by under the supervision of
....., has been examined by the committee and accepted
as an undergraduate capstone project in terms of scope and quality.

Prof. Dr. Jury Member Name Surname

Supervisor

.....

Prof. Dr. Jury Member Name Surname

Member

.....

Assoc. Prof. Dr. Jury Member Name Surname

Member

.....

Asst. Prof. Dr. Name Surname

Head of Department

.....

ACKNOWLEDGEMENTS

The authors gratefully acknowledge the guidance of the project advisor and the support of the campus transportation office during requirements elicitation and pilot evaluation. We also thank participating students and drivers for their constructive feedback throughout sprint reviews.

ABSTRACT (LOCAL LANGUAGE VERSION)

This study presents the design and implementation of a Scrum-driven Minimum Viable Product (MVP) for a Smart Campus Shuttle Management System. The project addresses inefficiencies in campus transportation, including uncertain waiting times, limited visibility of vehicle occupancy, and weak demand forecasting. The proposed solution integrates a mobile interface for passengers, a web dashboard for operators, and a backend service that provides real-time shuttle tracking and route analytics. The development process is organized into iterative sprints, and system requirements are derived from student, driver, and operations-office stakeholder needs. The MVP includes core modules such as user authentication, live shuttle map, estimated time of arrival, occupancy reporting, and incident notifications. Initial evaluation indicates improvements in user-perceived reliability and reduced average waiting uncertainty.

Keywords: Smart campus, Scrum, MVP, Transportation systems

ABSTRACT

This report presents the design and implementation of a Scrum-based Minimum Viable Product (MVP) for a Smart Campus Shuttle Management System. The project targets recurring operational challenges in campus transportation, including uncertain waiting times, low route transparency, and limited occupancy awareness. The proposed system consists of a student mobile application, an operator web panel, and a backend analytics service that provides real-time vehicle tracking, estimated time of arrival, and route performance metrics. Requirements were elicited from key stakeholders and translated into prioritized user stories for incremental sprint execution. The MVP delivers essential functionality, including secure login, live shuttle location, occupancy status updates, and service disruption alerts. Evaluation through scenario-based tests and pilot feedback suggests that the solution improves transportation visibility and decision support for both users and administrators.

Keywords: Scrum, MVP, Smart transportation, Software engineering

TABLE OF CONTENTS

	<u>Sayfa</u>
TABLE OF CONTENTS	iv
LIST OF TABLES.....	vi
LIST OF FIGURES	vii
LIST OF SYMBOLS AND ABBREVIATIONS	viii
1. Introduction	1
1.1. Problem Statement	1
1.2. Project Objectives	1
1.3. Project Scope	1
1.4. Report Organization	1
2. General Concepts	2
2.1. Domain Definition	2
2.2. Technologies Used.....	2
2.3. Fundamental Concepts	2
3. Related Work	3
3.1. Existing Systems	3
3.2. Academic Studies.....	3
3.3. Comparative Analysis	3
4. Software Process Model	4
4.1. Selected Software Development Model	4
4.2. Definition of Scrum Roles	4
4.3. Scrum Process	4
4.4. Scrum Implementation in the Project.....	4
5. Requirements Analysis	5
5.1. Stakeholders	5
5.2. User Requirements	5
5.2.1. Functional Requirements	5
5.2.2. Non-Functional Requirements	5
5.3. System Specifications	5
6. System Modeling.....	6
6.1. Context Diagram.....	6
6.2. Use Case Diagram	6
6.3. Use Case Descriptions.....	6
6.4. Sequence Diagrams.....	6
7. System Architecture and Design	7

7.1. System Architecture	7
7.2. Structural Models	7
7.2.1. Class Diagram	7
7.2.2. Inheritance (Generalization)	7
7.2.3. Aggregation / Composition	7
7.3. Behavioral Models	7
7.3.1. Event-Driven Model	8
7.3.2. State Diagram	8
8. System Implementation	9
8.1. Development Environment	9
8.2. System Components	9
8.3. Code Structure	9
9. MVP Development	10
9.1. MVP Definition	10
9.2. Implemented Features	10
9.3. System Interfaces	10
9.4. System Demonstration	10
10. Testing and Evaluation	11
10.1. Testing Strategy	11
10.2. Test Scenarios	11
10.3. Test Results	11
11. Results and Discussion	12
12. Conclusion and Future Work	13
12.1. Conclusion	13
12.2. Future Work	13
13. Appendices	16
13.1. Appendix A: Source Code	16
13.2. Appendix B: Sprint Backlog	16
13.3. Appendix C: User Guide	16

LIST OF TABLES

Sayfa

LIST OF FIGURES

Sayfa

LIST OF SYMBOLS AND ABBREVIATIONS

3GPP : 3rd Generation Partnership Project
ACB : Access Class Barring

1. Introduction

1.1. Problem Statement

Campus transportation services often operate with limited real-time visibility. Students do not know shuttle arrival times precisely, drivers lack structured incident reporting tools, and administrators have insufficient analytics for route optimization. As a result, waiting uncertainty, uneven vehicle utilization, and avoidable delays persist.

1.2. Project Objectives

The primary objective is to develop a practical and scalable MVP for smart shuttle operations. Specific objectives are: (i) provide real-time shuttle tracking to passengers, (ii) support operational monitoring through an administrative dashboard, and (iii) establish a data foundation for continuous route improvement.

1.3. Project Scope

The project focuses on a single-campus environment and covers core transportation workflows: route tracking, ETA estimation, occupancy status, and incident notification. The scope excludes advanced forecasting models, inter-city integration, and payment processing, which are planned for future phases.

1.4. Report Organization

Section 2 introduces domain concepts and enabling technologies. Section 3 reviews existing systems and academic studies. Sections 4–6 present process, requirements, and modeling artifacts. Sections 7–10 detail architecture, implementation, MVP outcomes, and testing. Section 11 discusses findings, and Section 12 concludes with future work.

2. General Concepts

2.1. Domain Definition

The project is positioned in intelligent transportation systems for higher-education campuses. This domain combines real-time mobility management, user-centric service design, and data-informed operational decision making.

2.2. Technologies Used

The solution stack includes a cross-platform mobile client (React Native), a web-based operator interface (React), backend services (Node.js and Express), and a relational database (PostgreSQL). Real-time updates are handled through WebSocket channels, and geospatial visualization is supported by map APIs.

2.3. Fundamental Concepts

Key concepts include Scrum-based iterative development, RESTful service design, event-driven updates, location-aware computing, and non-functional quality attributes such as reliability, responsiveness, and scalability.

3. Related Work

3.1. Existing Systems

Commercial and university-specific transit applications commonly provide static route maps and approximate schedules. However, many systems lack reliable occupancy indicators and actionable analytics for operational planning.

3.2. Academic Studies

Prior studies emphasize that real-time passenger information improves perceived service quality and reduces waiting anxiety. Research also shows that agile delivery models are effective for transportation software where requirements evolve based on user feedback.

3.3. Comparative Analysis

Compared with existing alternatives, the proposed MVP emphasizes three differentiators: (i) unified passenger and operator workflows, (ii) sprint-driven feature prioritization tied to measurable outcomes, and (iii) architecture readiness for future predictive analytics integration.

4. Software Process Model

4.1. Selected Software Development Model

A Scrum-based agile model was selected to accommodate evolving requirements and frequent stakeholder feedback. Two-week sprints were used to deliver incremental and testable product capabilities.

4.2. Definition of Scrum Roles

The Product Owner was represented by the campus transportation coordinator, the Scrum Master by the project lead, and the Development Team by software engineering students responsible for mobile, backend, and web modules.

4.3. Scrum Process

Each sprint followed planning, daily stand-up synchronization, sprint review, and retrospective practices. The product backlog was maintained as user stories with acceptance criteria and priority values.

4.4. Scrum Implementation in the Project

The project was completed in four sprints: Sprint 1 (core architecture and authentication), Sprint 2 (live shuttle tracking), Sprint 3 (occupancy and notifications), and Sprint 4 (operator dashboard and stabilization).

5. Requirements Analysis

5.1. Stakeholders

Primary stakeholders are students (passengers), shuttle drivers, and transportation office operators. Secondary stakeholders include IT administrators and faculty members who rely on predictable mobility services.

5.2. User Requirements

User requirements were gathered through short interviews and scenario walkthroughs. The requirements were translated into backlog items and validated at sprint reviews.

5.2.1. Functional Requirements

FR-1: Users shall view real-time shuttle locations on a map.

FR-2: Users shall receive estimated arrival times for selected stops.

FR-3: Operators shall monitor active routes and incident reports.

FR-4: Drivers shall submit route incidents through a simplified interface.

5.2.2. Non-Functional Requirements

NFR-1: The system shall respond to core requests within acceptable interactive latency under normal load.

NFR-2: The platform shall maintain high availability during service hours.

NFR-3: Access control shall enforce role-based permissions for student, driver, and operator roles.

5.3. System Specifications

The backend exposes REST endpoints for transactional operations and WebSocket events for live updates. Data entities include user, route, vehicle, stop, trip status, occupancy snapshot, and incident log.

6. System Modeling

6.1. Context Diagram

The context model includes three external actors—Student, Driver, and Operator—interacting with the Shuttle Management Platform. Third-party map and notification services are integrated as external systems.

6.2. Use Case Diagram

Major use cases are: Authenticate User, Track Shuttle, View ETA, Report Incident, Update Occupancy, and Monitor Operations.

6.3. Use Case Descriptions

Each use case defines actor, preconditions, main flow, alternative flows, and postconditions. For example, *Track Shuttle* requires authenticated access and returns location/ETA updates in near real time.

6.4. Sequence Diagrams

The sequence flow for *View ETA* begins with mobile stop selection, continues with backend route-state computation, and concludes with ETA response and live update subscription.

7. System Architecture and Design

7.1. System Architecture

A layered client–server architecture was adopted. Mobile and web clients consume backend APIs, while real-time channels propagate location and status events. Persistence and analytics are handled in the database layer.

7.2. Structural Models

The static design includes domain entities for users, vehicles, routes, stops, trips, occupancy logs, and incident records.

7.2.1. Class Diagram

The class model establishes associations between *Route–Stop*, *Vehicle–Trip*, and *User–Role* abstractions. Aggregated entities are designed to reduce data duplication and preserve auditability.

7.2.2. Inheritance (Generalization)

The *User* base type is generalized into *StudentUser*, *DriverUser*, and *OperatorUser*, each extending role-specific behavior.

7.2.3. Aggregation / Composition

A *Route* aggregates *Stops*; a *Trip* composes time-bound telemetry and occupancy snapshots to ensure lifecycle consistency.

7.3. Behavioral Models

Behavioral design captures event-triggered status transitions for active trips and notifications.

7.3.1. Event-Driven Model

Vehicle location changes publish events to subscribed clients. Operators receive threshold-based alerts (e.g., delay beyond tolerance).

7.3.2. State Diagram

The trip state machine transitions across *Scheduled*, *Active*, *Delayed*, and *Completed* states, with explicit guards for incident escalation.

8. System Implementation

8.1. Development Environment

Implementation was performed using Git-based version control, issue tracking for backlog management, and containerized local services for consistent development and testing.

8.2. System Components

The MVP consists of four components: mobile client, operator web panel, backend API/realtime gateway, and relational data store.

8.3. Code Structure

The codebase is modularized by feature domains (auth, routes, telemetry, notifications) with clear separation between controller, service, and data-access layers.

9. MVP Development

9.1. MVP Definition

The MVP is defined as the smallest deployable system that delivers measurable value to students and operators through real-time visibility and operational monitoring.

9.2. Implemented Features

Implemented features include secure login, route listing, live map tracking, ETA view, occupancy indicator, incident reporting, and operational dashboard widgets.

9.3. System Interfaces

The student interface prioritizes quick stop selection and ETA visibility. The operator panel emphasizes route health, delay indicators, and incident queue management.

9.4. System Demonstration

A pilot demonstration was conducted with representative scenarios: peak-hour boarding, delayed vehicle notification, and operator-side incident resolution workflow.

10. Testing and Evaluation

10.1. Testing Strategy

Testing combines unit validation for core services, integration testing for API/database interactions, and scenario-based acceptance testing for end-to-end workflows.

10.2. Test Scenarios

Representative scenarios include login and role routing, live map refresh consistency, ETA accuracy under delay events, and incident propagation to operator dashboard.

10.3. Test Results

Results indicate that all critical user stories satisfy acceptance criteria in pilot conditions. Minor UI refinements were identified for accessibility and message clarity.

11. Results and Discussion

The developed MVP demonstrates that a Scrum-driven approach can produce a functional and operationally relevant campus transportation platform within a constrained project timeline. Stakeholder feedback highlights increased transparency for students and improved situational awareness for operators. The main limitation is the restricted pilot scale, which may not fully represent long-term demand variability.

12. Conclusion and Future Work

12.1. Conclusion

This project delivered a coherent software engineering pipeline from requirements elicitation to MVP validation. The resulting system addresses a practical campus problem and establishes a reusable baseline architecture for subsequent releases.

12.2. Future Work

Future work will focus on predictive ETA models, demand-aware route optimization, accessibility enhancements, and broader deployment across multiple campuses with comparative performance analysis.

REFERENCES

CURRICULUM VITAE

PERSONAL INFORMATION

Full Name:

Nationality:

Place and Date of Birth:

E-mail:

EDUCATION

Degree	Institution	Graduation Year
High School:	...	...
Undergraduate:	...	...
Graduate:	...	...

PROFESSIONAL EXPERIENCE

Institution	Year
Institution 1:	...
Institution 2:	...

AREA OF EXPERTISE

Field
Field 1
Field 2

FOREIGN LANGUAGES

Language	Proficiency Level
Language 1:	Level 1
Language 2:	Level 2

CERTIFICATES

Certificate Area	Issuing Institution	Year
Certificate 1	...	...
Certificate 2	...	...

OTHER RELEVANT QUALIFICATIONS

PUBLICATIONS

Publication 1

Publication 2

PRESENTATIONS

Presentation 1

Presentation 2

13. Appendices

13.1. Appendix A: Source Code

This appendix includes representative code excerpts for authentication, live telemetry handling, and operator dashboard endpoints.

13.2. Appendix B: Sprint Backlog

This appendix summarizes sprint goals, selected user stories, acceptance criteria, and completion status across all four sprints.

13.3. Appendix C: User Guide

This appendix provides quick-start instructions for student, driver, and operator roles, including login, key actions, and troubleshooting notes.